Python E-Course

Module 9 — Object-Oriented Programming

Detailed Notes

Module Overview

Level: Intermediate

Duration: ~2 weeks

Prerequisites: Modules 1-8

What You'll Learn

- Define classes and create instances.
- Use __init__ to set up state and instance methods.
- Use class vs instance attributes.
- Inherit, override, and use polymorphism.
- Apply encapsulation conventions.
- Implement dunder methods like __str__, __eq__, __len__.
- Use @property and @dataclass.

Lessons

#	Lesson	Duration
1	Classes and Instances	30 min
2	Methods and Attributes	35 min
3	Inheritance	35 min
4	Polymorphism & Duck Typing	30 min
5	Encapsulation	25 min
6	Dunder Methods	35 min
7	@property and @dataclass	35 min

Assessment

- Exercises - Quiz - Assignment - Mini Project

What's Next

Module 10: Modules & Packages.

Lesson 1: Classes and Instances

Module: 9 — OOP

Duration: 30 minutes

Difficulty: Intermediate

Learning Objectives

- Define a class with class.
- Create instances and store data in them.
- Use __init__ to initialize each new instance.

Prerequisites

- Modules 1-8. Especially Module 8 Lesson 4 (custom exceptions previewed classes).

1. Why This Matters

A class is a template for things that share structure and behavior — a Student knows its name and grades; an Order knows its items and total. Once you can think in objects, you'll design clearer programs and read libraries more easily.

2. Concept

2.1 Defining a class

```
class Dog:
    pass
```

class keyword, PascalCase name, indented body. pass means empty body.

2.2 Creating an instance

```
fido = Dog()
print(type(fido))     # <class '__main__.Dog'>
```

Dog() calls the class like a function — it returns a new instance.

2.3 Attributes

You can attach data to an instance:

```
fido = Dog()
fido.name = "Fido"
fido.age = 3
print(fido.name)
```

But this is fragile — every Dog you create starts blank. Use __init__.

2.4 __init__ — the initializer

__init__ runs automatically when you create an instance:

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

fido = Dog("Fido", 3)
print(fido.name, fido.age)
```

- self is the new instance — always the first parameter.
- Assigning to self.something stores data on the instance.

2.5 Instances are independent

```
a = Dog("Fido", 3)
b = Dog("Rex", 5)
print(a.name, b.name)   # Fido Rex
```

2.6 Class vs instance

The class is the template; an instance is one concrete thing built from it. Dog is the class; fido and rex are instances.

3. Code Examples

Example 1: Empty class with attributes attached after

```
class Point:
    pass

p = Point()
p.x = 3
p.y = 4
print(p.x, p.y)
```

Expected Output: 3 4

Example 2: Proper __init__

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

p = Point(3, 4)
print(p.x, p.y)
```

Expected Output: 3 4

Example 3: Two independent instances

```
class Dog:
    def __init__(self, name):
        self.name = name

a = Dog("Fido")
b = Dog("Rex")
```

```
print(a.name)
print(b.name)
a.name = "Spot"
print(a.name, b.name)
```

Expected Output:

```
Fido
Rex
Spot Rex
```

Example 4: Class for a real concept

```
class Book:
    def __init__(self, title, author, pages):
        self.title = title
        self.author = author
        self.pages = pages

books = [
    Book("1984", "Orwell", 328),
    Book("Brave New World", "Huxley", 311),
]
for b in books:
    print(f"{b.title} by {b.author} ({b.pages} pages)")
```

Expected Output:

```
1984 by Orwell (328 pages)
Brave New World by Huxley (311 pages)
```

4. Common Mistakes

Mistake	What Happens	Fix
Forgetting self in __init__	TypeError: missing argument	Add self first
Defining __init__ without colon	SyntaxError	Add :
Calling Dog.name instead of fido.name	AttributeError	Access from instance
Setting attribute outside __init__ for "default"	All instances share via class attribute trap	Initialize in __init__

5. Practice Tasks

- Define class Car: ... with __init__(self, make, model, year). Create two cars and print their fields.
- Define class Rectangle with __init__(self, w, h). Create one and print w * h.
- Make a list of 3 Book instances and loop over them, printing each title.

6. Key Takeaways

- class Name: defines a class; Name(...) creates an instance.
- __init__(self, ...) runs on creation; use self.x = ... to store data.

- Each instance has its own attributes.

7. What's Next

Methods — functions that belong to a class and operate on instances.

Lesson 2: Methods and Attributes

Module: 9 — OOP

Duration: 35 minutes

Difficulty: Intermediate

Learning Objectives

- Define and call instance methods.
- Distinguish instance attributes from class attributes.
- Know about @classmethod and @staticmethod.

Prerequisites

- Module 9 Lesson 1

1. Why This Matters

Storing data isn't enough — objects also need behavior. Methods are functions that live on a class, with access to instance state via self. This is the core OOP idea.

2. Concept

2.1 Defining methods

```
class Counter:
    def __init__(self):
        self.value = 0

    def increment(self):
        self.value += 1

    def reset(self):
        self.value = 0
```

Call them through an instance:

```
c = Counter()
c.increment()
c.increment()
print(c.value)   # 2
c.reset()
```

self is automatically passed by Python when you call c.increment().

2.2 Methods with parameters

```
class Bank:
    def __init__(self, balance):
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
```

```
    def withdraw(self, amount):
        if amount > self.balance:
            raise ValueError("overdraft")
        self.balance -= amount
```

2.3 Instance vs class attributes

- Instance attribute: set on self, unique per instance.
- Class attribute: set on the class itself, shared by all instances.

```
class Dog:
    species = "Canis lupus familiaris"   # class attribute

    def __init__(self, name):
        self.name = name                  # instance attribute

a = Dog("Fido")
b = Dog("Rex")
print(a.species, b.species)   # shared
print(a.name, b.name)         # individual
```

If you write a.species = "x", you create an instance attribute that shadows the class one — only on a.

2.4 Mutable class attributes (gotcha)

```
class Box:
    items = []         # SHARED among all Box instances!

a = Box()
b = Box()
a.items.append(1)
print(b.items)    # [1] — surprise
```

Fix: initialize mutable state in __init__:

```
class Box:
    def __init__(self):
        self.items = []
```

2.5 @classmethod

Receives the class instead of an instance as first arg. Used for alternative constructors:

```
class Date:
    def __init__(self, year, month, day):
        self.year, self.month, self.day = year, month, day

    @classmethod
    def from_string(cls, s):
        y, m, d = s.split("-")
        return cls(int(y), int(m), int(d))
```

```
d = Date.from_string("2026-05-23")
print(d.year, d.month, d.day)
```

2.6 @staticmethod

A function inside a class for grouping. No self, no cls.

```
class MathUtils:
    @staticmethod
    def square(x):
        return x * x

print(MathUtils.square(5))   # 25
```

Often a module-level function would be just as good — use @staticmethod only when the function belongs conceptually to the class.

3. Code Examples

Example 1: Method that mutates state

```
class Counter:
    def __init__(self):
        self.value = 0

    def increment(self, by=1):
        self.value += by

c = Counter()
c.increment()
c.increment(5)
print(c.value)
```

Expected Output: 6

Example 2: Class attribute

```
class Dog:
    species = "Canis familiaris"

    def __init__(self, name):
        self.name = name

a = Dog("Fido")
b = Dog("Rex")
print(a.species, b.species)
Dog.species = "Updated"
print(a.species, b.species)
```

Expected Output:

```
Canis familiaris Canis familiaris
Updated Updated
```

Example 3: Shared mutable trap

```
class Box:
    items = []   # DON'T do this

a = Box()
b = Box()
a.items.append("x")
print(b.items)   # also ['x']
```

Expected Output: ['x']

Example 4: Classmethod

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    @classmethod
    def origin(cls):
        return cls(0, 0)

p = Point.origin()
print(p.x, p.y)
```

Expected Output: 0 0

Example 5: Staticmethod

```
class TempConv:
    @staticmethod
    def c_to_f(c):
        return c * 9 / 5 + 32

print(TempConv.c_to_f(25))
```

Expected Output: 77.0

4. Common Mistakes

Mistake	What Happens	Fix
Forgetting self in method	TypeError	Add self as first param
Mutable class attribute	Shared state surprise	Initialize in __init__
Calling method as Class.method() without instance	TypeError (missing self)	Call on an instance, or use @classmethod/@staticmethod
Using cls and self inconsistently	Confusion	self for instance, cls for classmethod

5. Practice Tasks

- Add deposit(self, amount) and withdraw(self, amount) to a class Account with balance.
- Add a class attribute interest_rate = 0.05 to it and a method that applies interest.

- Add a @classmethod from_dict(cls, data) that builds an Account from {"balance": 100}.

6. Key Takeaways

- Methods are functions on classes; first parameter is self.
- Instance attributes are per-instance; class attributes are shared.
- Initialize mutable state in __init__, never as a class attribute.
- @classmethod for alternative constructors; @staticmethod for grouped utilities.

7. What's Next

Inheritance — building new classes by extending existing ones.

Lesson 3: Inheritance

Module: 9 — OOP

Duration: 35 minutes

Difficulty: Intermediate

Learning Objectives

- Subclass an existing class with class Sub(Base):.
- Override methods in a subclass.
- Call the parent implementation with super().

Prerequisites

- Module 9 Lesson 2

1. Why This Matters

Inheritance lets you say "X is like Y, plus these extras." Custom exceptions (Module 8) were your first inheritance. In a real codebase, GUI widgets inherit from Widget, payment models inherit from PaymentBase, and so on.

2. Concept

2.1 Subclassing

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        return "Some sound"

class Dog(Animal):
    def speak(self):
        return "Woof!"
```

Dog(Animal) means Dog inherits from Animal. Dog instances get Animal's __init__, speak, and any other method — except where Dog overrides them.

2.2 Overriding

Dog.speak() overrides Animal.speak(). Calling dog.speak() runs Dog's version.

2.3 super() — calling the parent

If you want to extend a parent method rather than replace it:

```
class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name)
        self.breed = breed
```

super().__init__(name) runs Animal's init, which sets self.name. Then Dog adds self.breed.

2.4 isinstance and issubclass

```
d = Dog("Fido", "Lab")
isinstance(d, Dog)       # True
isinstance(d, Animal)    # True
issubclass(Dog, Animal)  # True
```

2.5 Multiple inheritance (preview)

A class can inherit from multiple parents:

```
class A: ...
class B: ...
class C(A, B): ...
```

Powerful but easy to misuse. Prefer composition (Module 9 L5) over deep multi-inheritance hierarchies. We won't go deep here.

2.6 When to use inheritance

- "Cat is a Animal" — yes.
- "Car is a Engine" — no, a car has an engine. Use composition (a field).

Rule of thumb: inheritance for is-a, composition for has-a.

3. Code Examples

Example 1: Basic subclass

```
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        return "Some sound"

class Dog(Animal):
    def speak(self):
        return "Woof!"

class Cat(Animal):
    def speak(self):
        return "Meow"

for a in [Dog("Fido"), Cat("Whiskers")]:
    print(a.name, "says", a.speak())
```

Expected Output:

```
Fido says Woof!
Whiskers says Meow
```

Example 2: super() in __init__

```
class Vehicle:
    def __init__(self, wheels):
        self.wheels = wheels

class Car(Vehicle):
    def __init__(self, model):
        super().__init__(wheels=4)
        self.model = model

c = Car("Civic")
print(c.wheels, c.model)
```

Expected Output: 4 Civic

Example 3: Extending a method

```
class Greeter:
    def greet(self, name):
        return f"Hello, {name}"

class LoudGreeter(Greeter):
    def greet(self, name):
        return super().greet(name).upper() + "!"

g = LoudGreeter()
print(g.greet("Maya"))
```

Expected Output: HELLO, MAYA!

Example 4: isinstance

```
class Animal: pass
class Dog(Animal): pass

d = Dog()
print(isinstance(d, Dog))
print(isinstance(d, Animal))
print(isinstance(d, object))
```

Expected Output:

```
True
True
True
```

Explanation: All classes ultimately inherit from object.

4. Common Mistakes

Mistake	What Happens	Fix
Forgetting super().__init__() in subclass	Parent state never set	Call super in every subclass init
Deep inheritance trees	Hard to follow	Prefer composition

Inheritance for code reuse, not is-a	Confusing types	Use a helper function or composition
Calling super(Parent, self).method()	Old-style Python 2 syntax	In Python 3, just super().method()

5. Practice Tasks

- Create Person with name and Employee(Person) with extra salary. Use super() in init.
- Override a speak method in a Dog subclass of Animal.
- Check with isinstance that an Employee is also a Person.

6. Key Takeaways

- class Sub(Base): makes Sub inherit from Base.
- Override methods by defining them on the subclass.
- super() calls the parent's version.
- Use inheritance for "is-a"; use composition for "has-a".

7. What's Next

Polymorphism and duck typing — what inheritance gets you besides code reuse.

Lesson 4: Polymorphism & Duck Typing

Module: 9 — OOP

Duration: 30 minutes

Difficulty: Intermediate

Learning Objectives

- Use polymorphism to write code that works with many types.
- Understand duck typing — "if it walks like a duck...".
- Pick between abstract base classes (preview) and duck typing.

Prerequisites

- Module 9 Lesson 3

1. Why This Matters

Polymorphism is what makes inheritance useful: you can iterate a list of Animals and call .speak() on each, without caring whether each one is a Dog, Cat, or Cow. Once you internalize it, you stop writing big if isinstance(...) chains.

2. Concept

2.1 Polymorphism

The same call (.speak()) does different things depending on the object's type:

```
for a in [Dog(), Cat(), Cow()]:
    print(a.speak())
```

Each subclass implements speak() differently; the caller doesn't need to know which.

2.2 Duck typing

Python doesn't care about formal type relationships — only about whether the object has the methods you call:

> "If it walks like a duck and quacks like a duck, it's a duck."

```
class Duck:
    def quack(self):
        return "Quack"

class RubberToy:
    def quack(self):
        return "Squeak"

def make_quack(thing):
    return thing.quack()
```

```
print(make_quack(Duck()))
print(make_quack(RubberToy()))
```

No inheritance needed — both have quack(), so both work.

2.3 Why this is liberating

You don't need to write a base class everyone inherits from. Just call the methods you need. If the object provides them, your code works.

This is why Python file objects, BytesIO, StringIO, and many other things are interchangeable — they all support .read() and .write().

2.4 When to use inheritance vs duck typing

- Inheritance: when there's shared state or implementation to reuse.
- Duck typing: when classes are unrelated but should respond to the same calls.

2.5 Abstract base classes (brief)

If you want to enforce that subclasses implement certain methods, use abc:

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        ...

class Square(Shape):
    def __init__(self, side):
        self.side = side
    def area(self):
        return self.side ** 2
```

Trying to instantiate Shape() raises a TypeError. Useful in larger projects; usually overkill for small scripts.

3. Code Examples

Example 1: Polymorphic loop

```
class Animal:
    def speak(self):
        return "..."

class Dog(Animal):
    def speak(self):
        return "Woof"

class Cat(Animal):
    def speak(self):
        return "Meow"
```

```
for a in [Dog(), Cat(), Animal()]:
    print(a.speak())
```

Expected Output:

```
Woof
Meow
...
```

Example 2: Duck typing — no shared base

```
class Duck:
    def quack(self):
        return "Quack"

class Person:
    def quack(self):
        return "I'm quacking like a duck"

def make_it_quack(x):
    return x.quack()

print(make_it_quack(Duck()))
print(make_it_quack(Person()))
```

Expected Output:

```
Quack
I'm quacking like a duck
```

Example 3: Reuse via duck-typed interface

```
class WriteLogger:
    def __init__(self, target):
        self.target = target

    def log(self, message):
        self.target.write(message + "\n")

import io
buf = io.StringIO()
logger = WriteLogger(buf)
logger.log("hello")
print(buf.getvalue())
```

Expected Output: hello\n

Explanation: WriteLogger works with any object that has .write() — file, StringIO, network stream.

Example 4: Abstract base class

```
from abc import ABC, abstractmethod

class Shape(ABC):
```

```
    @abstractmethod
    def area(self):
        ...

class Circle(Shape):
    def __init__(self, r):
        self.r = r
    def area(self):
        return 3.14159 * self.r ** 2

try:
    Shape()
except TypeError as e:
    print("Cannot instantiate:", e)

print(Circle(2).area())
```

Expected Output:

```
Cannot instantiate: Can't instantiate abstract class Shape with abstract method
area
12.56636
```

4. Common Mistakes

Mistake	What Happens	Fix
Big isinstance chains	Brittle, anti-pattern	Add a method to each class and call it polymorphically
Forcing inheritance for unrelated types	Confusing hierarchy	Use duck typing
Forgetting abstract methods on subclass	TypeError at instantiation	Implement all @abstractmethods

5. Practice Tasks

- Write Animal, Dog, Cat, each with .speak(). Loop a list of them and call .speak().
- Write a function play(quacker) that calls quacker.quack(). Pass in two unrelated classes that both have .quack().
- Use abc.ABC to define Shape with abstract area, and create a Triangle subclass.

6. Key Takeaways

- Polymorphism: same call, different behavior per type.
- Duck typing: no shared base needed — just shared methods.
- Use ABCs (sparingly) when you want to enforce a contract.

7. What's Next

Encapsulation — how Python signals "this is private" and how _ and __ differ.

Lesson 5: Encapsulation

Module: 9 — OOP

Duration: 25 minutes

Difficulty: Intermediate

Learning Objectives

- Use _name to signal "internal" attributes.
- Use __name for name mangling (and know when not to).
- Prefer composition (has-a) over deep inheritance.

Prerequisites

- Module 9 Lesson 4

1. Why This Matters

Encapsulation is the practice of hiding internal details and exposing a clean interface. Python doesn't enforce privacy like Java does — it relies on convention. Learn the conventions and your code stays readable and refactorable.

2. Concept

2.1 Convention: _name

A leading underscore says "this is internal — don't touch from outside":

```
class Cache:
    def __init__(self):
        self._data = {}    # treat as private

    def get(self, key):
        return self._data.get(key)
```

Python won't stop you from accessing cache._data — but if you do, you accept that the maintainer might change it without notice.

2.2 Name mangling: __name

Two leading underscores trigger name mangling. The attribute is rewritten as _ClassName__name:

```
class A:
    def __init__(self):
        self.__secret = 42

a = A()
print(a._A__secret)   # works — but ugly
```

Use __ only when you really need to avoid name collisions in subclasses. Most code uses single _.

2.3 Composition over inheritance

Prefer to contain an object than to inherit from it:

```
# Inheritance (often wrong)
class Car(Engine): ...

# Composition (usually right)
class Car:
    def __init__(self):
        self.engine = Engine()
```

Composition is more flexible and easier to test.

2.4 Public interface

A class's public interface is everything not starting with _. Document it well; treat it as a contract. Internal stuff can change freely.

2.5 Read-only via property (preview)

For a value the user reads but can't directly set, use @property (Lesson 7).

3. Code Examples

Example 1: Private convention

```
class Account:
    def __init__(self, balance):
        self._balance = balance

    def deposit(self, amount):
        self._balance += amount

    def get_balance(self):
        return self._balance

a = Account(100)
a.deposit(50)
print(a.get_balance())
# a._balance += 1000   # legal but discouraged
```

Expected Output: 150

Example 2: Name mangling

```
class Vault:
    def __init__(self):
        self.__pin = 1234

v = Vault()
try:
    print(v.__pin)
except AttributeError as e:
    print("blocked:", e)
print(v._Vault__pin)
```

Expected Output:

```
blocked: 'Vault' object has no attribute '__pin'
1234
```

Example 3: Composition

```
class Engine:
    def start(self):
        return "vroom"

class Car:
    def __init__(self):
        self.engine = Engine()
    def start(self):
        return self.engine.start()

print(Car().start())
```

Expected Output: vroom

Example 4: Clean public interface

```
class TodoList:
    def __init__(self):
        self._items = []          # internal

    def add(self, item):
        self._items.append(item)

    def remove(self, item):
        self._items.remove(item)

    def items(self):
        return list(self._items)  # return a copy

todo = TodoList()
todo.add("groceries")
todo.add("emails")
print(todo.items())
```

Expected Output: ['groceries', 'emails']

Explanation: Returning a copy prevents callers from mutating internal state.

4. Common Mistakes

Mistake	What Happens	Fix
Using __ everywhere	Hard to subclass	Use _ unless you really need mangling
Exposing mutable internal state	Callers mutate your private list	Return a copy
Inheriting from a class just to reuse one method	Tangled hierarchy	Use composition or a helper

Treating Python's _ as enforced privacy	Surprised when callers access it	Document and trust the convention

5. Practice Tasks

- Build class Wallet with _balance. Expose deposit, withdraw, balance methods.
- Refactor a Car(Engine) design into Car containing an Engine instance.
- Add a tasks() method to a TodoList that returns a copy of internal list. Show mutating the returned list does not affect the internal one.

6. Key Takeaways

- _name = "internal" (convention only).
- __name triggers name mangling — use sparingly.
- Prefer composition (has-a) to inheritance.
- Return copies of mutable internal state if you want it really protected.

7. What's Next

Dunder methods — __str__, __eq__, __len__ — that make your classes feel native.

Lesson 6: Dunder Methods

Module: 9 — OOP

Duration: 35 minutes

Difficulty: Intermediate

Learning Objectives

- Implement __str__, __repr__ for printing.
- Implement __eq__, __lt__, __hash__ for comparisons.
- Implement __len__, __getitem__, __iter__ for container behavior.

Prerequisites

- Module 9 Lesson 5

1. Why This Matters

"Dunder" methods (double-underscore: __name__) let your class plug into Python's syntax: print(obj), obj == other, len(obj), for x in obj. Implementing them turns your class from data-bag into a first-class Python citizen.

2. Concept

2.1 __init__ — already known

Constructor. Covered in Lesson 1.

2.2 __str__ and __repr__

- __str__ — user-friendly string. Called by print() and str().
- __repr__ — unambiguous, developer-oriented. Called by REPL display and repr().

If you only define one, define __repr__; __str__ falls back to it.

```
class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __repr__(self):
        return f"Point({self.x}, {self.y})"
    def __str__(self):
        return f"({self.x}, {self.y})"

p = Point(3, 4)
print(p)        # (3, 4)         <- __str__
print(repr(p))  # Point(3, 4)    <- __repr__
```

2.3 Equality and ordering

- __eq__(self, other) for == (and != follows for free).
- __lt__(self, other) for < — and the others (__gt__, __le__, __ge__) often follow via functools.total_ordering.

```
class Money:
    def __init__(self, amount):
        self.amount = amount
    def __eq__(self, other):
        return isinstance(other, Money) and self.amount == other.amount
    def __lt__(self, other):
        return self.amount < other.amount

print(Money(100) == Money(100))   # True
print(Money(50) < Money(75))      # True
```

If you define __eq__, you should also define __hash__ (or set __hash__ = None to disallow hashing).

2.4 Length and containment

- __len__(self) for len(obj).
- __contains__(self, item) for item in obj.

```
class Deck:
    def __init__(self, cards):
        self.cards = cards
    def __len__(self):
        return len(self.cards)
    def __contains__(self, card):
        return card in self.cards

d = Deck(["A♠", "K♥"])
print(len(d))           # 2
print("A♠" in d)        # True
```

2.5 Iteration

- __iter__(self) — return an iterator (often iter(self.some_list)).
- __getitem__(self, i) — indexing, also gives default iteration.

```
class Deck:
    def __init__(self, cards):
        self.cards = cards
    def __iter__(self):
        return iter(self.cards)
    def __getitem__(self, i):
        return self.cards[i]

d = Deck(["A♠", "K♥", "Q♦"])
for c in d:
    print(c)
print(d[1])
```

2.6 Other useful dunders

Dunder	Triggers
__add__	a + b
__bool__	bool(a) / if a:

__call__	a(...) (makes the instance callable)
__enter__/__exit__	with a: (context manager)

You don't need to implement all of these. Pick the ones that make your class easier to use.

3. Code Examples

Example 1: str vs repr

```
class Book:
    def __init__(self, title, author):
        self.title, self.author = title, author
    def __repr__(self):
        return f"Book({self.title!r}, {self.author!r})"
    def __str__(self):
        return f"{self.title} by {self.author}"

b = Book("1984", "Orwell")
print(b)
print(repr(b))
```

Expected Output:

```
1984 by Orwell
Book('1984', 'Orwell')
```

Example 2: Equality

```
class Pt:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __eq__(self, other):
        return isinstance(other, Pt) and self.x == other.x and self.y == other.y

print(Pt(1,2) == Pt(1,2))
print(Pt(1,2) == Pt(3,4))
```

Expected Output:

```
True
False
```

Example 3: len + contains

```
class WordList:
    def __init__(self, words):
        self.words = words
    def __len__(self):
        return len(self.words)
    def __contains__(self, w):
        return w in self.words

wl = WordList(["apple", "banana"])
print(len(wl))
print("apple" in wl)
print("cherry" in wl)
```

Expected Output:

```
2
True
False
```

Example 4: Iteration

```
class CountUp:
    def __init__(self, n):
        self.n = n
    def __iter__(self):
        return iter(range(1, self.n + 1))

for i in CountUp(5):
    print(i)
```

Expected Output:

```
1
2
3
4
5
```

Example 5: Add and bool

```
class Money:
    def __init__(self, amount):
        self.amount = amount
    def __add__(self, other):
        return Money(self.amount + other.amount)
    def __bool__(self):
        return self.amount > 0
    def __repr__(self):
        return f"Money({self.amount})"

print(Money(100) + Money(50))
print(bool(Money(0)))
print(bool(Money(5)))
```

Expected Output:

```
Money(150)
False
True
```

4. Common Mistakes

Mistake	What Happens	Fix
Returning non-string from __str__/__repr__	TypeError	Always return a string
Implementing __eq__ without __hash__	Instance becomes unhashable	Define both, or __hash__ = None deliberately

__eq__ not checking isinstance(other, ...)	Money() == "100" returns weird results	Always type-check other
Too many dunders	Class becomes obscure	Only add what callers need

5. Practice Tasks

- Add __str__ to a Book class showing "Title by Author".
- Add __eq__ to a Point class and compare two equal points.
- Make a Bag class wrapping a list; add __len__, __contains__, __iter__.

6. Key Takeaways

- Dunders connect your class to Python syntax.
- __repr__ should be unambiguous; __str__ user-friendly.
- Pair __eq__ with __hash__.
- Don't over-engineer — implement only the dunders that improve usage.

7. What's Next

The last lesson: @property for computed/validated attributes, and @dataclass for boilerplate-free classes.

Lesson 7: @property and @dataclass

Module: 9 — OOP

Duration: 35 minutes

Difficulty: Intermediate

Learning Objectives

- Use @property for computed and validated attributes.
- Use @dataclass to remove boilerplate from data-holding classes.

Prerequisites

- Module 9 Lesson 6

1. Why This Matters

@property lets you turn methods into attribute-like access — no get_balance() everywhere. @dataclass generates __init__, __repr__, __eq__ for you. Combined, they cut huge amounts of boilerplate.

2. Concept

2.1 @property — read-only computed attribute

```
class Circle:
    def __init__(self, radius):
        self.radius = radius

    @property
    def area(self):
        return 3.14159 * self.radius ** 2

c = Circle(5)
print(c.area)    # access like an attribute, no parens
# c.area = 100   # AttributeError — no setter defined
```

2.2 Property with setter (validation)

```
class Account:
    def __init__(self, balance):
        self._balance = balance

    @property
    def balance(self):
        return self._balance

    @balance.setter
    def balance(self, value):
        if value < 0:
            raise ValueError("balance cannot be negative")
        self._balance = value
```

```
a = Account(100)
a.balance = 50         # uses setter
# a.balance = -10      # ValueError
```

2.3 @dataclass — auto-generated boilerplate

```
from dataclasses import dataclass

@dataclass
class Point:
    x: int
    y: int
```

Equivalent to writing __init__, __repr__, and __eq__ manually.

```
p = Point(3, 4)
print(p)               # Point(x=3, y=4)
print(p == Point(3, 4))  # True
```

2.4 Dataclass options

```
from dataclasses import dataclass, field
from typing import List

@dataclass
class Order:
    id: int
    items: List[str] = field(default_factory=list)
    discount: float = 0.0
```

- Defaults work like normal __init__ defaults.
- field(default_factory=list) avoids the mutable-default trap.
- frozen=True makes instances immutable: @dataclass(frozen=True).

2.5 When to use each

- @property when you want method-backed attribute access.
- @dataclass when your class is mostly data and you want less boilerplate.
- Plain class when you have rich behavior or unusual patterns.

You can mix them — a dataclass with custom methods and properties is fine.

3. Code Examples

Example 1: Read-only property

```
class Square:
    def __init__(self, side):
        self.side = side

    @property
    def area(self):
        return self.side ** 2

s = Square(4)
```

```
print(s.area)
try:
    s.area = 100
except AttributeError as e:
    print("blocked:", e)
```

Expected Output:

```
16
blocked: can't set attribute 'area'
```

Example 2: Validating setter

```
class Temperature:
    def __init__(self, c):
        self.celsius = c

    @property
    def celsius(self):
        return self._c

    @celsius.setter
    def celsius(self, value):
        if value < -273.15:
            raise ValueError("below absolute zero")
        self._c = value

    @property
    def fahrenheit(self):
        return self._c * 9/5 + 32

t = Temperature(25)
print(t.celsius, t.fahrenheit)
try:
    t.celsius = -500
except ValueError as e:
    print("error:", e)
```

Expected Output:

```
25 77.0
error: below absolute zero
```

Example 3: Basic dataclass

```
from dataclasses import dataclass

@dataclass
class Book:
    title: str
    author: str
    pages: int = 0

b = Book("1984", "Orwell", 328)
```

```
print(b)
print(b == Book("1984", "Orwell", 328))
```

Expected Output:

```
Book(title='1984', author='Orwell', pages=328)
True
```

Example 4: Dataclass with mutable default

```
from dataclasses import dataclass, field
from typing import List

@dataclass
class Cart:
    items: List[str] = field(default_factory=list)

a = Cart()
b = Cart()
a.items.append("apple")
print(a.items)
print(b.items)
```

Expected Output:

```
['apple']
[]
```

Explanation: Each Cart gets its own list — no shared-state bug.

Example 5: Frozen dataclass

```
from dataclasses import dataclass

@dataclass(frozen=True)
class Point:
    x: int
    y: int

p = Point(1, 2)
try:
    p.x = 99
except Exception as e:
    print("blocked:", e)
```

Expected Output: blocked: cannot assign to field 'x'

4. Common Mistakes

Mistake	What Happens	Fix
Calling obj.area() instead of obj.area for a property	TypeError (calling an int)	Drop the parens
Mutable defaults in dataclass: items: list = []	Shared between instances	Use field(default_factory=list)

Inheriting from dataclass without re-decorating	Subclass missing autogen methods	Decorate the subclass too
Property without setter and trying to assign	AttributeError	Define @x.setter if you need writes

5. Practice Tasks

- Add @property full_name to a Person(first, last) that returns "first last".
- Make Temperature.celsius validated: reject below -273.15.
- Turn a 5-field Book class into a @dataclass.

6. Key Takeaways

- @property makes methods feel like attributes; add a @x.setter for writes.
- @dataclass autogenerates __init__, __repr__, __eq__.
- Use field(default_factory=list) for mutable defaults.
- @dataclass(frozen=True) makes instances immutable.

7. What's Next

End of Module 9. Next: Module 10 — Modules & Packages, where we split code across files and projects.

Module 9 — Exercises

21 exercises (3 per lesson).

9.1 Classes

- (E) Define class Cat with __init__(self, name). Create 2 cats; print names.
- (M) Define Rectangle(w, h). Compute w * h outside the class.
- (H) Create a list of 4 Book(title, author) instances; print each.

9.2 Methods/Attributes

- (E) Add bark() to Dog returning "Woof!". Print on a Dog.
- (M) Add deposit/withdraw to Account. Test both.
- (H) Add @classmethod from_string(cls, s) to a class.

9.3 Inheritance

- (E) Define Animal and subclass Dog; override .speak().
- (M) Define Employee(Person) adding salary. Use super().
- (H) Use isinstance to test that an Employee is a Person.

9.4 Polymorphism

- (E) Loop [Dog(), Cat()] and call .speak().
- (M) Write play(quacker) that calls quacker.quack(). Pass 2 unrelated classes.
- (H) Use abc.ABC to enforce area() on Shape subclasses.

9.5 Encapsulation

- (E) Add _balance to Wallet; expose via balance() method.
- (M) Refactor Car(Engine) to Car containing Engine (composition).
- (H) Return a copy of an internal list from a method; show mutating it doesn't affect internal state.

9.6 Dunders

- (E) Add __str__ to Book.
- (M) Add __eq__ to Point and compare equal points.
- (H) Add __len__, __contains__, __iter__ to a Bag wrapping a list.

9.7 Property/Dataclass

- (E) Add @property full_name to Person(first, last).
- (M) Add a validated setter for age (≥0) on a Person.
- (H) Convert a 5-field Book to a @dataclass.

Module 9 — Quiz

10 MCQs.

Q1

What's the first parameter of an instance method? A. cls B. self C. this D. nothing

Answer: B (L1)

Q2

What does __init__ do? A. Defines a class B. Runs once when a class is defined C. Runs when an instance is created D. Runs when the program starts

Answer: C (L1)

Q3

Where should mutable defaults live? A. As class attributes B. As __init__ parameter defaults C. Set on self inside __init__ D. As module globals

Answer: C (L2)

Q4

What's the right way to call a parent's __init__? A. Parent.__init__(self, ...) B. super().__init__(...) C. Parent(...) D. Both A and B work; B is preferred

Answer: D — both work but super() is idiomatic. (L3)

Q5

Duck typing means: A. All objects must inherit from Duck B. Python checks types at runtime C. If it has the methods you call, it works D. There's no inheritance

Answer: C (L4)

Q6

A leading single underscore (_x) means: A. Private — enforced by Python B. Internal — convention only C. Class attribute D. Static

Answer: B (L5)

Q7

Which dunder is called by print(obj)? A. __repr__ B. __str__ C. __print__ D. __display__

Answer: B (with __repr__ as fallback). (L6)

Q8

If you implement __eq__ you should also: A. Implement __hash__ (or set it to None) B. Implement __lt__ C. Inherit from Equal D. Nothing else

Answer: A (L6)

Q9

What does @property do? A. Makes a method callable B. Turns a method into attribute-like access C. Makes a class immutable D. Auto-generates __init__

Answer: B (L7)

Q10

What does @dataclass generate? A. __init__, __repr__, __eq__ B. __str__, __hash__ C. Just __init__ D. Custom validation

Answer: A (L7)

Module 9 — Assignment: Library System

Estimated time: 2-2.5 hours

Combines: Module 9 + Modules 4, 7, 8

Brief

Model a small library: Book, Member, Library. Members can borrow and return books; library tracks who has what. Persist to JSON between runs.

Requirements

- Book dataclass with fields: id (int), title, author, year, available: bool = True.
- Member class:
- __init__(self, id, name).
- _borrowed: list[int] — book IDs currently borrowed.
- borrowed_ids() returns a copy of the list.
- Custom exceptions (inherit from LibraryError):
- BookNotFound, MemberNotFound, BookUnavailable, LimitReached.
- Library class:
- add_book(book), add_member(member).
- borrow(member_id, book_id) — checks availability and member limit (max 3 books). Raises appropriate exception.
- return_book(member_id, book_id).
- find_books(query) — case-insensitive search on title or author.
- to_dict() and from_dict(data) for JSON round-trip.
- save(path) / load(path) classmethods.
- __str__ and __repr__ on Book and Member.
- A main() that demos: creates a library, adds 3 members and 5 books, performs several borrow/return operations (some failing), saves, prints status.

Constraints

- Modules 1-9 only.
- Use @dataclass for Book.
- Use custom exceptions for all error paths.
- Persist with json + pathlib.

Expected Output (sample)

```
== Library demo ==
[ok] alice borrowed 'Brave New World'
[err] BookUnavailable: 'Brave New World' is not available
[ok] alice returned 'Brave New World'
[err] LimitReached: bob has already borrowed 3 books
Saved library.json
```

Grading Rubric

Criterion	Weight
Classes correct	30%
Exceptions used	15%
Borrow/return logic	25%
JSON persistence	15%
Style + dunders	15%

Submission

Save as assignment_module9.py.

Module 9 — Mini Project: Pet Shop Simulator

Overview

Model a small pet shop: an Animal base class with concrete subclasses (Dog, Cat, Parrot), a Shop containing animals, and a CLI to interact with it. Practice inheritance, polymorphism, and encapsulation.

Concepts Used

- Classes, inheritance, polymorphism (M9 L1-L4)
- Encapsulation (M9 L5)
- Dunder methods (M9 L6)
- Functions, error handling (M4, M8)

Features

- Animal base with name, age, price; abstract sound() method.
- Subclasses Dog, Cat, Parrot each implementing sound().
- Shop with add(animal), remove(name), find(name), inventory().
- CLI: add / list / sound NAME / remove NAME / quit.

Starter Code

```
# Mini Project: Pet Shop Simulator
from abc import ABC, abstractmethod
from dataclasses import dataclass

class Animal(ABC):
    def __init__(self, name, age, price):
        self.name = name
        self.age = age
        self.price = price

    @abstractmethod
    def sound(self):
        ...

    def __str__(self):
        return f"{type(self).__name__}({self.name}, age {self.age}, ₹{self.price})"

class Dog(Animal):
    def sound(self):
        return "Woof!"

class Cat(Animal):
    def sound(self):
        return "Meow"

class Parrot(Animal):
    def sound(self):
        return "Hello!"
```

```
class Shop:
    def __init__(self):
        self._animals = []

    def add(self, animal):
        self._animals.append(animal)

    def remove(self, name):
        for a in self._animals:
            if a.name == name:
                self._animals.remove(a)
                return True
        return False

    def find(self, name):
        for a in self._animals:
            if a.name == name:
                return a
        return None

    def inventory(self):
        return list(self._animals)

    def total_value(self):
        return sum(a.price for a in self._animals)

KINDS = {"dog": Dog, "cat": Cat, "parrot": Parrot}

def run():
    shop = Shop()
    while True:
        cmd = input("\n[add | list | sound NAME | remove NAME | total | quit] >
").strip().split(maxsplit=1)
        if not cmd:
            continue
        match cmd[0].lower():
            case "add":
                kind = input("kind (dog/cat/parrot): ").strip().lower()
                if kind not in KINDS:
                    print("unknown kind"); continue
                name = input("name: ")
                age = int(input("age: "))
                price = float(input("price: "))
                shop.add(KINDS[kind](name, age, price))
                print("added.")
            case "list":
                for a in shop.inventory():
                    print(a)
            case "sound":
                a = shop.find(cmd[1])
                print(a.sound() if a else "no such animal")
            case "remove":
```

```
                print("removed" if shop.remove(cmd[1]) else "no such animal")
            case "total":
                print(f"₹{shop.total_value():.2f}")
            case "quit":
                print("Bye!"); break
            case _:
                print("unknown")

if __name__ == "__main__":
    run()
```

Expected Interaction

```
[add | list | ...] > add
kind: dog
name: Fido
age: 3
price: 5000
added.
[add | list | ...] > add
kind: parrot
name: Polly
age: 1
price: 1200
added.
[add | list | ...] > list
Dog(Fido, age 3, ₹5000.0)
Parrot(Polly, age 1, ₹1200.0)
[add | list | ...] > sound Polly
Hello!
[add | list | ...] > total
₹6200.00
[add | list | ...] > quit
Bye!
```

Extension Challenges

- Add a Fish subclass.
- Persist shop state to JSON between runs.
- Add a feed NAME method that decreases the animal's hunger; raises FeedError when over-fed.

Grading Rubric

Criterion	Weight
Animal hierarchy correct	25%
sound() is polymorphic	15%
Shop encapsulates animals	15%
CLI all commands work	25%
Style + dunders	20%